

NeuroSem: Final Research and Claude Code Implementation Specification

Executive decision

Project: NeuroSem, a software framework and empirical study for detecting silent behavioral changes in computational neuron models after they are edited, translated, refactored, or repaired.

Central research question: Can a computational neuron model remain structurally valid, execute successfully, and pass one conventional reference test while behaving differently under other scientifically relevant electrical stimuli?

Proposed contribution: NeuroSem represents each model using a multi-stimulus electrophysiological response fingerprint, creates controlled model mutations, and learns a small battery of stimulation protocols that detects behavior-changing transformations efficiently. Existing NeuroML tools already validate model structure, execute simulations, compare behavior across simulators, and support biological model testing; NeuroSem must reuse those capabilities rather than claim them as new.^{[1][2][^3]}

Primary scientific claim to test, not assume: A compact, mutation-calibrated perturbation battery detects more non-equivalent transformations than schema validation, successful execution, or a single canonical stimulation protocol.

Role of Claude Code: Claude Code is primarily a development assistant. It may later be evaluated in a small, isolated case study as one source of model transformations, but it must not define the title, main hypothesis, or principal novelty.

Best IEEE target if the completed results are methodologically strong: *IEEE/ACM Transactions on Computational Biology and Bioinformatics* (TCBB), because its stated scope emphasizes algorithmic, mathematical, statistical, and computational methods central to computational biology. A future IEEE computational-intelligence-in-bioinformatics conference may also fit

because past IEEE CIBCB calls explicitly included modeling, simulation, and optimization of biological systems, but the current call, deadlines, indexing terms, and costs must be verified before submission.^{[4][5]}

Plain-language project

A published computer model of a neuron is given an electrical input and produces a voltage trace. A researcher or software tool then edits the model. The edited file may still be legal, may run without crashing, and may reproduce the expected response to one standard input. Nevertheless, the edit may have changed how the model responds to a weaker current, a slowly increasing current, a long pulse, or release from a negative current.

NeuroSem tests the neuron under several electrical conditions instead of trusting one successful run. It extracts interpretable measurements—such as spike count, firing rate, first-spike timing, action-potential width, adaptation, and rebound behavior—and compares the original and transformed models. It then searches for the smallest combination of stimulation protocols that exposes the largest number of meaningful hidden changes.

The anticipated practical message is simple:

A model that runs is not necessarily a model that retained its scientific behavior, and one reference trace may not be enough to establish preservation.

IEEE feasibility

Can a high-school student submit?

An IEEE paper is not submitted through a general application to IEEE. The author selects a particular IEEE journal or conference and submits a manuscript through that venue's submission system. IEEE's journal process instructs authors to choose a journal whose aims and scope fit the work, follow that journal's author instructions, and submit through the IEEE Publishing Portal; it also warns that an out-of-scope manuscript can be rejected before peer review.^[^6]

The reviewed IEEE submission guidance specifies requirements concerning originality, scope, formatting, authorship, and peer review rather than an academic-degree requirement. No promise of eligibility for every venue should be inferred: the final venue’s current author instructions must be checked directly, and a parent, school, or affiliated adult may be needed for contracts, payment, travel, or presentation logistics if the submitting author is legally a minor.^{[7][6]}

The work must be the author’s original contribution and cannot be under simultaneous review elsewhere. IEEE states that submitted work must not already have appeared as a publication or be under review at another publication.^{[8][7]}

Suitable IEEE pathways

Venue	Fit	What the paper would need	Main concern
IEEE/ACM TCBB	Best journal fit	A substantial computational method, rigorous held-out evaluation, useful biological findings, and reproducible software	A software demonstration without algorithmic or biological depth may be insufficient
IEEE CIBCB-type conference	Good potential conference fit	A completed, concise experimental paper on computational modeling, optimization, or computational intelligence in biology	The 2027 call, deadlines, format, registration, and indexing must be checked when released
IEEE Journal of Biomedical and Health Informatics	Conditional	A clear biomedical-informatics platform or health-relevant translation, not only model testing	Its emphasis includes informatics, modeling, and computational biology, but the manuscript needs a stronger health/informatics connection ^[^9]

IEEE TNSRE	Weak-to-conditional	Direct relevance to neural engineering, rehabilitation, stimulation, or assistive technology	Its stated focus is neural systems and rehabilitation engineering, so generic cellular-model validation may be out of scope ^{[10][11]}
IEEE Access	Broad fallback	Technically sound original work across an IEEE field	Fully open-access publication may involve substantial fees; current charges and waiver rules must be checked before submission ^[12]

Recommended publication sequence

1. Build and validate the two-model pilot.
2. Complete a documented novelty review.
3. Expand only if the pilot confirms hidden behavior-changing transformations.
4. Freeze the study design and run the held-out experiment.
5. Prepare a full paper for TCBB or a suitable computational-biology journal.
6. Consider an IEEE conference only when a current call clearly matches the work and the deadlines are feasible.
7. Do not submit the same or substantially similar manuscript to a journal and conference simultaneously.^{[13][7]}

IEEE permits preprint posting under specified conditions, including on arXiv or TechRxiv, but the exact policy and copyright notice requirements should be rechecked at the time of posting.^{[14][15]}

Scientific background

Computational neuron models

Conductance-based neuron models use differential equations to represent changes in membrane voltage and ion-channel currents. Their behavior depends on parameters such as maximal conductances, reversal potentials, membrane capacitance, channel kinetics, morphology, stimulation, and numerical settings.

Different parameter combinations can generate similar observed activity. Prior neuron-model research has quantified model behavior using multiple stimulation protocols—including sag, spike, threshold, rheobase, frequency-current, and current-voltage protocols—rather than relying on one response. This supports the biological premise that multi-condition behavior is informative, but it also means NeuroSem cannot claim to invent multi-protocol characterization.^[16]

NeuroML and LEMS

NeuroML is a standardized ecosystem for representing and exchanging computational neuroscience models. Its current ecosystem includes validation tools, simulation support, databases, model-validation tools, and SciUnit integration.^[3]

LEMS supplies machine-readable component and simulation descriptions. In practical terms, the NeuroML files describe the model, while a LEMS simulation file specifies what to run, for how long, at what time step, and what variables to record.

The official documentation shows that `pynml` and `jnml` can validate NeuroML files, and `pyNeuroML` exposes programmatic validation functions. `jNeuroML` can execute LEMS simulations directly, while `pyNeuroML` can invoke `jNeuroML` from Python and can also generate and run code for NEURON.^{[17][1]}

Existing validation

NeuroML already distinguishes multiple notions of validity: schema and logical correctness, consistent execution, cross-simulator behavior, and agreement with experimentally observed features. Open Source Brain Model Validation can define model tests and run them locally or in continuous integration.^{[2][3]}

Therefore, the project must not claim novelty for:

- Validating NeuroML syntax.
- Checking units and references.
- Running NeuroML models.
- Comparing a saved trace or spike data.
- Executing tests in GitHub Actions.
- Testing a model against biological observations.

These become baselines or dependencies.

Feature extraction

The Electrophys Feature Extraction Library, eFEL, extracts standardized features from simulated or experimental voltage traces, including action-potential width and amplitude. NeuroSem should reuse eFEL for supported features and record its version, interpolation settings, stimulation windows, threshold settings, and any undefined-feature behavior because these choices can alter extracted values.^{[18][19][20]}

Metamorphic testing

Metamorphic testing is useful when the correct output for an individual test is difficult to specify. It checks necessary relationships between related inputs and outputs instead of requiring a complete answer oracle.^{[21][22]}

Examples relevant to NeuroSem include:

- Expressing the same physical quantity in equivalent units should preserve behavior.
- Correctly renaming identifiers and references should preserve behavior.
- Extending a deterministic simulation should preserve the overlapping trace interval.
- Refining the numerical time step should approach a stable result.

Metamorphic testing has already been proposed for simulation verification and validation, so its generic use is not itself novel.[²¹]

Mutation testing

Mutation testing introduces controlled changes and asks whether a test suite detects them. In NeuroSem, a mutation is a deliberate change to a neuronal model, stimulus, reference, or numerical setting. A behavior-changing mutation that survives ordinary checks but fails under an additional protocol is the phenomenon of interest.

Mutation testing has also been used to evaluate reproducibility safeguards in machine-learning repositories. NeuroSem must therefore contribute domain-specific mutation operators, electrophysiological response analysis, protocol selection, and evidence about neuronal-model transformation integrity.[²³]

Novelty boundary

Already established

- Standardized neuron-model representation and simulation.
- Structural and logical validation.
- Cross-simulator model testing.
- Electrophysiological feature extraction.
- Multi-protocol neuron characterization.
- Metamorphic testing of simulations.
- Mutation testing of software and scientific workflows.
- Generic test-suite minimization and coverage optimization.

Candidate novel contribution

A mutation-calibrated framework that represents transformed conductance-based neuron models using perturbation fingerprints and selects compact stimulation

batteries for detecting changes hidden from structural, execution, and canonical-regression checks.

Candidate empirical discoveries

- Some transformations preserve a canonical response but alter behavior under other stimuli.
- Particular protocols are disproportionately effective for particular mutation families.
- A small selected battery outperforms a cost-matched random battery on models excluded from protocol selection.
- Behavior-changing mutations can be organized into interpretable response-drift classes.
- Some AI-assisted transformations exhibit the same failure modes as controlled mutations.

These are hypotheses. They must not be written as findings before data collection.

Required novelty sweep

Before claiming novelty, search at minimum:

- Google Scholar.
- PubMed.
- IEEE Xplore.
- ACM Digital Library.
- Web of Science or Scopus if accessible.
- Semantic Scholar.
- arXiv and bioRxiv.
- GitHub, PyPI, and Zenodo.

Use combinations of:

- conductance neuron model metamorphic testing
- NeuroML mutation testing

- neuron model semantic regression
- behavior preservation neuronal model conversion
- perturbation fingerprint neuron model
- automatic stimulation protocol selection model validation
- mutation testing computational neuroscience
- scientific software testing NeuroML
- AI coding agent neuron simulation
- LLM computational neuroscience code

Create docs/novelty_matrix.csv with columns for citation, year, model type, mutations, multiple stimuli, electrophysiology features, protocol optimization, held-out evaluation, AI transformations, software availability, and distinction from NeuroSem. The present sweep found closely related foundations but did not establish an existing project with the complete proposed combination; absence from a search is not proof that no such work exists.^{[2][16][3][21]}

Formal study design

Operational definitions

Reference model: An unmodified, validated, reproducibly executable neuron model with recorded provenance.

Transformation: Any deliberate alteration to the model repository, model description, simulation protocol, or execution configuration.

Valid transformation: A representational or engineering change intended to preserve tested scientific behavior, such as equivalent-unit conversion or consistent identifier renaming.

Mutant: A transformed model created by exactly one recorded mutation operator.

Admissible non-equivalent mutant: A structurally valid, executable mutant with reproducible behavioral divergence from its reference somewhere in a predefined exhaustive protocol battery.

Canonical protocol: The single standard stimulation test used as the conventional behavioral-regression baseline.

Perturbation fingerprint: The vector of electrophysiological measurements generated by a model across multiple stimulation protocols.

Silent semantic drift: Reproducible behavioral divergence that passes structural validation, executes successfully, and passes the canonical regression test but is exposed by a held-out or expanded perturbation test.

Empirical semantic certificate: A finite set of passing protocol-feature tests. This is evidence of tested preservation, not a proof of equivalence under every possible input.

Fingerprint definition

For model m , protocol p , and feature f , define the fingerprint as the collection:

$$F(m) = \{f(m, p) : p \in P, f \in E\}.$$

P is the protocol set and E is the feature set. A transformation is detected when at least one prespecified protocol-feature difference exceeds its calibrated tolerance.

Research questions

RQ1: How often do structurally valid, executable mutations survive a canonical stimulation test but differ under other stimuli?

RQ2: Does a compact optimized protocol battery detect more admissible mutants than the canonical protocol at a comparable computational cost?

RQ3: Does the optimized battery outperform randomly selected, cost-matched protocol sets?

RQ4: Does the selected battery generalize to base models excluded from protocol selection?

RQ5: Does it generalize to at least one excluded mutation family?

RQ6: Which protocols and electrophysiological features detect each mutation family?

RQ7: Do AI-assisted transformations produce silent drift detectable by the frozen framework?

Primary hypothesis

A compact protocol battery selected on discovery models will detect a higher proportion of admissible non-equivalent mutants on held-out models than a canonical single-protocol regression test.

Primary endpoint

Held-out mutant detection rate under the selected NeuroSem battery versus canonical regression.

Secondary endpoints

- Detection rate versus cost-matched random protocol batteries.
- Silent-survival rate after canonical testing.
- False-positive rate on valid transformations.
- Detection rate by mutation family.
- Protocols required to reach increasing levels of exhaustive-battery coverage.
- Runtime and simulations per detected mutant.
- Agent-task success under basic versus NeuroSem validation.

Solo-feasible scope

Pilot

Component	Pilot target
Reference models	2
Candidate protocols	4
Extracted features	3-5

Mutation families	3
Mutants per model	5–10
Valid transformations	4–6
Simulator	jNeuroML only

The pilot succeeds if at least one admissible mutation passes the canonical protocol and is reproducibly detected by another protocol, feature extraction is stable under numerical refinement, and valid transformations do not trigger widespread false positives.

Full solo study

Component	Target range
Reference models	12–16
Discovery models	8–10
Held-out models	4–6
Candidate protocols	10–16
Mutation families	6–8
Admissible mutants	100–160
Valid transformations	24–40
Claude Code tasks	20–30
Cross-simulator models	4–6, optional but valuable

These numbers are planning bounds, not a substitute for a pilot-based runtime estimate or power analysis. Reduce mutant count before sacrificing curation, provenance, controls, or held-out evaluation.

Model selection

Inclusion criteria

A model must:

- Have a traceable public source.
- Have a license permitting the intended use and redistribution, or be handled without redistribution if the license requires it.
- Pass current NeuroML validation.
- Execute deterministically in the frozen environment.
- Complete the candidate protocols within a practical runtime.
- Produce interpretable voltage output.
- Respond meaningfully to current injection.
- Have scientific provenance linking it to a publication or established repository.
- Avoid unavailable proprietary dependencies.

Diversity goals

Prefer a small set spanning several response behaviors:

- Tonic spiking.
- Spike-frequency adaptation.
- Bursting.
- Rebound firing.
- Sag response.
- Distinct excitability thresholds.

Begin with single-compartment or simple models. Add only a small number of multicompartment models after the pipeline is stable.

Provenance record

For each model record:

- Internal ID.
- Model name.
- Scientific citation.
- Source URL.
- Download date.
- License and license URL.
- Original files.
- SHA-256 hashes.
- Required includes.
- Simulator version.
- Known expected behavior.
- Inclusion or exclusion decision and reason.

Protocol battery

Initial protocols

1. Zero-current baseline.
2. Weak depolarizing step.
3. Rheobase search.
4. Step at a fixed multiple of model-specific rheobase.
5. Long suprathreshold step.
6. Depolarizing ramp.
7. Hyperpolarizing step.
8. Hyperpolarization-release rebound test.
9. Short suprathreshold pulse.

10. Paired pulses.
11. Deterministic chirp, only if implementation is stable.
12. Frozen pseudo-random waveform, only as a later extension.

Model-specific rheobase normalization is important because the same absolute current can be negligible for one model and overwhelming for another.

Initial features

- Baseline voltage.
- Steady-state voltage.
- Voltage deflection.
- Sag ratio.
- Rheobase.
- Spike count.
- Mean firing frequency.
- First-spike latency.
- Action-potential amplitude.
- Action-potential half-width.
- After-hyperpolarization depth.
- First and last interspike intervals.
- Adaptation index.
- Burst count.
- Qualitative firing regime.

Use eFEL only where the feature is well-defined for the protocol. Record missing or undefined features explicitly rather than replacing them with zero.^{[19][20]}

Transformations and mutations

Valid transformations

These measure false positives:

- Equivalent physical-unit conversion.
- XML formatting or comments.
- Numerically equivalent literal formatting.
- Identifier renaming with every reference updated.
- File factoring with references preserved.
- Reordering semantically independent elements.
- Replacing an inherited default with the same explicit value.

Each valid transformation must be verified using structural inspection and the exhaustive high-resolution protocol battery before inclusion.

Mutation families

Stimulus mutations

- Change input amplitude.
- Change onset.
- Change duration.
- Change total simulation time.
- Record the wrong variable.

Biophysical mutations

- Scale maximal conductance.
- Change reversal potential.
- Change membrane capacitance.
- Scale a channel time constant.

- Change initial voltage.
- Apply a channel to the wrong segment group.

Reference mutations

- Reference the wrong channel definition.
- Omit a required include.
- Duplicate a conductance assignment.
- Point to a wrong but dimensionally compatible component.

Numerical mutations

- Increase integration time step.
- Alter solver configuration.
- Reduce spatial discretization.
- Change recording resolution enough to corrupt extracted features.

Use single-fault mutants in the primary study. Compound faults belong only in a secondary stress test.

Mutant classification

Classify every generated mutant as:

1. Structurally invalid.
2. Structurally valid but non-executable.
3. Executable but numerically unstable.
4. Executable and behaviorally equivalent within the tested domain.
5. Executable and non-equivalent.
6. Silent under canonical testing but detected by another protocol.

Only categories 5 and 6 enter the main scientific detection-rate denominator. Crashes and schema errors may be reported separately but must not inflate NeuroSem’s central effectiveness result.

Tolerance calibration

Perfect numerical equality is not expected across every execution path. For each reference model and protocol:

1. Run at nominal time step h .
2. Repeat at $h/2$.
3. Repeat at $h/4$ when practical.
4. Interpolate traces to a common time grid.
5. Compare trace and feature differences.
6. Repeat any stochastic protocol with fixed seeds.
7. Define feature-specific absolute and relative tolerance floors.
8. Freeze tolerances before held-out evaluation.

A conceptual rule is:

$$\tau_{m,p,f} = \max(\tau_f^{abs}, r_f |f(m, p)|, c |f_h(m, p) - f_{h/2}(m, p)|).$$

The constants must be justified from discovery/reference runs and sensitivity analysis. Do not adjust them to improve held-out performance.

Protocol selection

Construct a binary matrix in which each row is an admissible discovery mutant, each column is a protocol, and each cell records whether that protocol detects the mutant.

Use greedy maximum coverage as the transparent baseline:

1. Select the protocol detecting the most currently undetected discovery mutants.
2. Mark those mutants covered.

3. Select the protocol detecting the largest number of remaining mutants.
4. Continue until reaching budget k .
5. Freeze the selected set.
6. Evaluate it on held-out models and mutation families.

Compare against:

- Canonical protocol alone.
- Random sets with the same protocol count.
- Random sets with comparable runtime.
- Full candidate battery.

An optional cost-sensitive objective may maximize coverage while penalizing simulation runtime. Do not introduce a complicated machine-learning model unless it clearly improves held-out performance and interpretability.

Train-test separation

Discovery set

May be used to:

- Develop mutation operators.
- Calibrate tolerances.
- Select protocols.
- Debug feature extraction.
- Choose the primary analysis.

Held-out model set

Must not influence protocol selection or threshold adjustment.

Held-out mutation family

At least one mutation family should be excluded from protocol selection and used only in final evaluation. This is harder than a random mutant split and better tests generalization.

Leakage controls

- Store held-out manifests in a separate path.
- Prevent selection code from reading held-out labels.
- Freeze split files with hashes.
- Log every run touching held-out data.
- Do not repeatedly inspect held-out failures and redesign around them.
- Use a second final lockbox only if enough models remain.

Statistical analysis

The same mutants are evaluated by several validation strategies, creating paired binary outcomes. Report transparent paired counts and differences rather than only an aggregate accuracy.

Primary analysis:

- Detection rate for selected battery and canonical regression on held-out mutants.
- Paired difference in detection rate.
- Confidence interval obtained by resampling at the base-model level.
- Exact paired test if assumptions and sample size support it.

Secondary analysis:

- Selected versus random cost-matched batteries.
- Mutation-family-specific detection.
- False-positive rates on valid transformations.

- Coverage versus number of protocols.
- Runtime-adjusted coverage.
- Robustness to stricter and looser tolerances.

Mutants derived from one base model are correlated, so uncertainty estimation should resample base models or otherwise account for clustering. Do not treat every mutant as fully independent.

Preregister:

- Primary hypothesis.
- Primary endpoint.
- Inclusion and exclusion rules.
- Model and mutation splits.
- Canonical protocol.
- Candidate protocols.
- Tolerance policy.
- Primary statistical comparison.
- Handling of undefined features.
- Treatment of crashes and equivalent mutants.

Claude Code experiment

Position in manuscript

Use a secondary subsection titled **Application to AI-assisted neuronal-model transformation**. Do not include Claude in the paper title unless the paper's actual purpose changes into a product benchmark.

Task types

- Repair a seeded unit error.
- Restore a changed stimulus protocol.
- Rename a channel safely.
- Refactor included model files.
- Convert a quantity to an equivalent unit.
- Change one specified conductance while preserving all other parameters.
- Repair a channel reference.
- Improve runtime without changing tested outputs.
- Diagnose why a schema-valid model changed firing behavior.

Isolation protocol

- Freeze NeuroSem before agent testing.
- Give the agent only public tests.
- Hide selected perturbation protocols and answer keys.
- Start every trial from the same clean commit.
- Use fresh, isolated sessions.
- Record Claude Code version, underlying model identifier, access date, permissions, budget, prompts, transcripts, patches, logs, and costs.
- Prohibit manual intervention during an autonomous trial.
- Score outputs with deterministic hidden tests.
- Audit every patch manually after automatic scoring.

Permitted claim

Under the evaluated Claude Code configuration, a specified fraction of transformations passed basic checks but failed frozen perturbation tests.

Prohibited claim

Coding agents generally cannot be trusted in computational neuroscience.

The latter would require multiple agents, versions, task distributions, and much broader evidence.

AI-assisted development policy

Claude Code may help build the repository, but the human author must control scientific ground truth, experimental design, thresholds, exclusions, interpretation, and final claims.

For IEEE submissions, substantive use of generative AI should be disclosed. Current IEEE conference guidance states that only humans may be authors, substantive AI use should be disclosed, and the human authors remain responsible for accuracy, originality, and integrity.^[^24]

Maintain `AI_USE_LOG.md` with:

- Date and time.
- Tool and model identifier.
- Task requested.
- Prompt or transcript path.
- Files changed.
- Human review performed.
- Tests executed.
- Problems found.
- Whether the output was accepted, modified, or rejected.

Suggested disclosure:

Claude Code was used to assist with software scaffolding, debugging, refactoring, test implementation, and documentation. All generated code was reviewed by the author and

evaluated through deterministic unit and integration tests, controlled mutations, numerical-convergence checks, and reproduction of reference simulations. Scientific ground truth, inclusion criteria, tolerance calibration, held-out evaluation, statistical analysis, and interpretation remained under the author’s control. In a separately identified experiment, Claude Code was evaluated as a coding agent using frozen prompts, isolated sessions, fixed permissions, and hidden deterministic evaluation procedures.

Repository architecture

```
neurosem/  
├── README.md  
├── LICENSE  
├── CITATION.cff  
├── pyproject.toml  
├── environment.yml  
├── Dockerfile  
├── Makefile  
├── .github/workflows/ci.yml  
├── configs/  
│   ├── study.yaml  
│   ├── tolerances.yaml  
│   ├── features.yaml  
│   └── agent_policy.yaml  
├── data/  
│   ├── model_manifest.csv  
│   ├── protocol_manifest.csv  
│   ├── mutation_manifest.csv  
│   ├── valid_transforms.csv  
│   └── splits/  
├── models/  
│   ├── raw/  
│   ├── curated/  
│   └── snapshots/  
├── src/neurosem/  
│   ├── cli.py  
│   ├── schemas.py  
│   ├── provenance.py  
│   ├── simulators/  
│   │   └── base.py
```

```
| | | └─ jneuroml.py
| | | └─ neuron.py
| | └─ protocols/
| | | └─ definitions.py
| | | └─ generate.py
| | | └─ rheobase.py
| | └─ features/
| | | └─ efel_adapter.py
| | | └─ trace_metrics.py
| | | └─ regimes.py
| | └─ mutations/
| | | └─ base.py
| | | └─ stimulus.py
| | | └─ biophysics.py
| | | └─ references.py
| | | └─ numerical.py
| | └─ transforms/
| | | └─ units.py
| | | └─ identifiers.py
| | | └─ factoring.py
| | └─ validation/
| | | └─ structural.py
| | | └─ execution.py
| | | └─ canonical.py
| | | └─ fingerprint.py
| | | └─ convergence.py
| | └─ selection/
| | | └─ matrix.py
| | | └─ greedy.py
| | | └─ splits.py
| | └─ experiments/
| | | └─ pilot.py
| | | └─ discovery.py
| | | └─ heldout.py
| | | └─ agent.py
| | └─ analysis/
| | | └─ metrics.py
| | | └─ bootstrap.py
| | | └─ figures.py
| └─ tests/
| | └─ unit/
| | └─ integration/
| | └─ regression/
```

```
|   └─ fixtures/
|─ workflows/
|   └─ run_reference.py
|   └─ generate_mutants.py
|   └─ calibrate_tolerances.py
|   └─ select_protocols.py
|   └─ evaluate_heldout.py
|─ results/
|   └─ raw/
|   └─ processed/
|   └─ tables/
|   └─ figures/
|─ docs/
|   └─ glossary.md
|   └─ novelty_matrix.csv
|   └─ model_selection.md
|   └─ protocol_catalog.md
|   └─ mutation_catalog.md
|   └─ statistical_plan.md
|   └─ ai_disclosure.md
|─ manuscript/
|   └─ manuscript.md
|   └─ supplement.md
```

Data requirements

Every simulation must record:

- Unique run ID.
- Model ID and file hash.
- Source-model snapshot.
- Protocol ID.
- Mutation or transformation ID.
- Simulator and version.
- Time step and duration.
- Random seed, if relevant.

- Container/environment digest.
- Execution status.
- Runtime.
- Trace path and hash.
- Feature-table path and hash.
- Timestamp.
- Git commit.

Raw results must be immutable. Derived results should be reproducible from raw outputs with one workflow command.

Suggested commands:

```
neurosem validate-models
neurosem run-reference
neurosem calibrate-tolerances
neurosem generate-mutants
neurosem classify-mutants
neurosem build-fingerprints
neurosem select-protocols --budget 4
neurosem evaluate-heldout
neurosem evaluate-agent
neurosem analyze
neurosem reproduce-paper
```

Implementation milestones

Milestone 0: Audit

Deliver:

- Dependency and version audit.
- License audit.
- Name-conflict search.

- Prior-art matrix.
- Risk register.
- Decision log.

Stop before implementation and request approval.

Milestone 1: Environment

Deliver:

- Python package.
- Frozen dependencies.
- jNeuroML execution.
- eFEL installation.
- Docker environment.
- Continuous integration.
- Two tiny fixture models.

Exit criterion: one command validates and simulates both fixture models on a clean installation.

Milestone 2: Reference pipeline

Deliver:

- Trace format.
- Provenance records.
- Feature extraction.
- Deterministic rerun test.
- Basic plots for manual inspection.

Exit criterion: repeated runs produce the same conclusions within predeclared numerical tolerance.

Milestone 3: Protocol engine

Deliver:

- Weak and strong steps.
- Rheobase search.
- Long step.
- Ramp.
- Hyperpolarization and rebound.

Exit criterion: protocol timing and current amplitudes are independently tested and visually verified.

Milestone 4: Mutation engine

Deliver:

- Three pilot mutation families.
- One-change-per-mutant enforcement.
- Machine-readable provenance.
- Structural-validity and execution classification.

Exit criterion: at least 20 manually audited mutants match their labels.

Milestone 5: Fingerprints

Deliver:

- eFEL adapter.
- Trace metrics.
- Convergence calibration.

- Per-mutant diagnostic reports.
- Detection matrix.

Exit criterion: every detected mutant is linked to the protocol, feature, threshold, and evidence trace that caused detection.

Milestone 6: Pilot decision

Run two to six models and 20–60 mutants. Continue only if canonical testing misses at least some reproducible non-equivalent mutants and additional protocols detect them without unacceptable false positives.

If no hidden drift exists, do not manufacture it. Reframe the project as an empirical evaluation of the adequacy of existing validation or stop.

Milestone 7: Selection and holdouts

Deliver:

- Frozen discovery and held-out splits.
- Greedy coverage selection.
- Random cost-matched baselines.
- Leakage tests.

Exit criterion: selection code cannot access held-out labels.

Milestone 8: Frozen full study

Deliver:

- Preregistration.
- Frozen configurations and hashes.
- Full execution logs.
- Statistical analysis.

- Robustness and ablation analyses.

Milestone 9: Agent study

Deliver:

- Frozen prompts.
- Isolated Claude Code trials.
- Raw transcripts and patches where redistribution is permitted.
- Hidden evaluation results.
- Manual patch audit.

Milestone 10: Release and manuscript

Deliver:

- Public repository.
- Versioned software release.
- DOI-minting archive.
- Reproduction guide.
- Manuscript and supplement.
- Data, code, and AI-use statements.
- Venue-specific IEEE formatting only after choosing the target.

Required controls

- **No-change controls:** formatting or documentation edits must preserve behavior.
- **Valid-transformation controls:** equivalent units and correct renames estimate false positives.
- **Deterministic mutation controls:** known changes verify validator sensitivity.
- **Canonical baseline:** measures what one ordinary reference protocol detects.

- **Random battery baseline:** determines whether optimization adds value.
- **Exhaustive empirical battery:** identifies non-equivalence within the tested domain.
- **Numerical-refinement control:** separates scientific change from solver error.
- **Held-out models:** tests generalization.
- **Held-out mutation family:** tests robustness beyond known faults.
- **Agent isolation:** prevents Claude from seeing hidden evaluators.

Required figures

1. **Concept figure:** original and transformed models look similar under the canonical protocol but diverge under another stimulus.
2. **Validation cascade:** total mutants, schema-valid mutants, executable mutants, canonical survivors, perturbation detections, and unresolved survivors.
3. **Detection heat map:** mutants by protocols, grouped by model and mutation family.
4. **Efficiency curve:** detection rate versus protocol count for canonical, random, selected, and exhaustive batteries.
5. **Held-out generalization:** discovery versus unseen models and unseen mutation family.
6. **False-positive analysis:** valid transformations and tolerance sensitivity.
7. **Case studies:** two or three scientifically interpretable examples.
8. **AI case study:** Claude Code task outcomes by validation layer, clearly secondary.

Success criteria

The project earns a strong paper only if it demonstrates most of the following:

- Hidden, reproducible drift exists in more than isolated contrived examples.
- Drift includes scientifically interpretable feature or firing-regime changes, not only tiny trace deviations.
- The selected battery materially outperforms canonical testing.

- The selected battery outperforms cost-matched random selection.
- Results generalize to models excluded from selection.
- False positives on valid transformations are low and transparently reported.
- Conclusions survive reasonable tolerance changes.
- Results do not depend entirely on Claude Code errors.
- Code and data reproduce the paper's tables and figures.

Failure and pivot criteria

Pivot or stop if:

- Existing validation detects nearly every meaningful mutation.
 - Most mutations only crash or violate schema.
 - Selected protocols do not beat random protocols on held-out models.
 - False positives remain high.
 - The result depends on one unstable model.
 - Findings vanish under numerical refinement.
-

References

1. [Validating NeuroML Models](#) - Using the command line tools#. Both pynml (provided by pyNeuroML) and jnml (provided by jNeuroML) ca...
2. [GitHub - OpenSourceBrain/osb-model-validation: Tools for automated model validation in OpenSourceBrain projects](#) - Tools for automated model validation in OpenSourceBrain projects - OpenSourceBrain/osb-model-validat...
3. [The NeuroML ecosystem for standardized multi-scale modeling in neuroscience](#) - Data-driven models of neurons and circuits are important for understanding how the properties of mem...

4. [CIBCB 2024 - IEEE CIBCB and BBTC - 21st IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology 27-29...](#)
5. [CSDL Pub Home: Transactions on Computational Biology and Bioinformatics](#) - Emphasizing the algorithmic, mathematical, statistical, and computational methods that are central i...
6. [The IEEE Article Submission Process - IEEE Author Center Journals](#) - Outlines the steps for the IEEE Article Submission Process. Includes tips for selecting the right IE...
7. [Authors Rights and Responsibilities](#) - This page is provided to help authors stay informed of the most recent policy matters that will have...
8. [An FAQ on Intellectual Property Rights for IEEE Authors](#), [An FAQ on Intellectual Property Rights for IEEE Authors](#)
9. [Scope of J-BHI - Journal of Biomedical and Health Informatics \(JBHI\)](#) - This section focuses on informatics and engineering methods that acquire, process, interpret, and ap...
10. [Editorial Policy - Transactions on Neural Systems and Rehabilitation Engineering \(TNSRE\)](#) - The IEEE Transactions on Neural Systems and Rehabilitation Engineering is one of the leading researc...
11. [Submission Guidelines - Transactions on Neural Systems and Rehabilitation Engineering \(TNSRE\)](#)
12. [IEEE Access](#) - IEEE Access is a multidisciplinary, all-electronic archival journal, presenting the results of origi...
13. [Submission Policies](#) - The material is original. IEEE policy does not allow simultaneous submission of similar manuscripts ...
14. [IEEE Preprint Policy](#) - Bottom line: it is fine to post to TechRxiv or arXiv with certain conditions. The PSPB Operations Ma...
15. [General and Author Policies - 2026 IEEE International Conference ...](#)

16. [Associating changes in output behavior with changes in parameter values in spiking and bursting neuron models](#) - Several recent studies have demonstrated that neuronal models allow multiple parameter value solutio...
17. [Simulating NeuroML Models](#)
18. [eFEL Documentation](#)
19. [Release 5.7.34 BBP, EPFL](#) - eFEL is an electrophysiology feature extraction library that allows neuroscientists automatically ex...
20. [eFeature descriptions¶](#)
21. [Metamorphic Testing on the Continuum of Verification ... - NIST](#)
22. [Metamorphic Testing: A Review of Challenges and Opportunities](#) - Metamorphic testing is an approach to both test case generation and test result verification. A cent...
23. [Mutation Testing for Reproducibility Safeguards inMachine Learning...](#)
24. [Call For Papers - FOCS 2026 - IEEE Computer Society](#)